DOSSIER TECHNIQUE

Bernard Quizz

Application web de quiz multijoueur en temps réel

Auteur	Kalandy (bthiery)
Programme	ADMIN1 - Administration IT
Période	Mars 2026
Version	1.0.0 - Production
Hébergement	Alwaysdata (mutualisé)
URL	bernardquizz.alwaysdata.net
Technologies	PHP 8.4, MariaDB 11.4, JavaScript

1. Présentation du Projet

Bernard Quizz est une application web de quiz multijoueur en temps réel. Elle permet à un hôte de créer une partie, de la configurer, puis de la faire rejoindre par des joueurs via un simple code PIN à 6 chiffres depuis n'importe quel appareil disposant d'un navigateur - sans installation.

Le projet a été développé de A à Z dans le cadre du programme scolaire ADMIN1, en couvrant l'ensemble de la chaîne : conception, développement, sécurisation et déploiement en environnement de production.

1.1 Concept de jeu

- Un hôte crée un salon depuis son compte, choisit un quiz et un mode de jeu
- Les joueurs rejoignent via le code PIN affiché à l'écran, personnalisent leur avatar "Bernard"
- L'hôte lance la partie - les questions s'affichent sur tous les écrans simultanément
- Les réponses sont envoyées en temps réel, les scores mis à jour à chaque manche
- Un podium animé avec effets de confettis clôture chaque partie

1.2 Trois modes de jeu

Mode	Principe	Spécificités
🏆 **Classique**	Points basés sur la vitesse de réponse (500 à 1000 pts)	Bonus streak +200 pts · Dernière question × 2
⚔ **Battle Royale**	Le dernier du classement est éliminé à chaque manche	Un seul survivant · Idéal entre amis
♡ **Survie**	Chaque joueur dispose de 3 cœurs	Erreur = -1 cœur · Zéro cœur = éliminé

2. Stack Technique

Composant	Technologie	Justification du choix
Backend	PHP 8.4	Langage maîtrisé, natif sur alwaysdata, adapté au développement web rapide
Base de données	MariaDB 11.4 (MySQL)	Inclus dans l'offre hébergement, relationnel, idéal pour les données structurées
Frontend	HTML5 · CSS3 · JavaScript Vanilla	Pas de dépendance framework lourd, performance maximale côté client
CSS Framework	Tailwind CSS (CDN)	Prototypage rapide par classes utilitaires, responsive natif
Typographie	Google Fonts - Caveat	Police manuscrite unique renforçant l'identité visuelle "jeu"
Temps réel	Polling HTTP toutes les	WebSockets non disponibles en

	1,5s	hébergement mutualisé - solution pragmatique
Sécurité mots de passe	bcrypt (password_hash)	Algorithme de hachage adaptatif recommandé par PHP et OWASP
Accès base de données	PDO + requêtes préparées	Abstraction DB + protection native contre les injections SQL
Hébergement	Alwaysdata (mutualisé)	Disponible dans le cadre scolaire, PHP + MySQL + HTTPS inclus
Assets avatars	Open-source pinknose.me	Système de couches multicouches libre de droits, personnalisable

3. Architecture de l'Application

3.1 Vue d'ensemble

Le projet suit une architecture MVC légère sans framework. Chaque fichier PHP gère à la fois la logique métier et la vue. La communication en temps réel repose sur un fichier central api_live.php qui lit et écrit des fichiers JSON dans un dossier /sessions/, servant de bus de données partagé entre tous les clients.

3.2 Structure des fichiers

Fichier	Rôle
index.php	Page d'accueil publique - saisie du code PIN · connexion · inscription
login.php	Authentification - vérification bcrypt, création de session PHP
register.php	Inscription - validation, hachage mot de passe, insertion en DB
dashboard.php	Menu principal - affichage adapté selon le rôle utilisateur
lobby.php	Personnalisation complète de l'avatar Bernard avant de rejoindre
host_game.php	Création du salon - choix du mode, génération PIN, salle d'attente
host_screen.php	Écran hôte - pilotage de la partie, timer, classement, podium

play.php	Écran joueur mobile - questions, boutons réponse, tchat, score
game_screen.php	Variante écran joueur (accès via URL directe)
api_live.php	API centrale - gestion de l'état de partie par polling HTTP
profil.php	Profil utilisateur - statistiques, rang, quêtes, Bernard favori
manage_quizzes.php	Bibliothèque de quiz - affichage, création, duplication, suppression
edit_quiz.php	Éditeur de questions - CRUD complet sur les questions d'un quiz
admin_users.php	Panel admin - gestion des rôles utilisateurs (admin/fondateur)
db.php	Connexion PDO centrale + fonction utilitaire hasRole()
documentation.php	Documentation utilisateur intégrée au site
mentions_legales.php	Mentions légales, CGU et politique de données
sessions/PIN.json	État partagé d'une partie - lu/écrit par api_live.php

3.3 Flux de déroulement d'une partie

Cycle de statuts dans le fichier JSON de session

lobby → reveal → playing → show_answer → leaderboard → (boucle) → finished
Chaque statut est déclenché par l'hôte via api_live.php. Les clients (joueurs et hôte) interrogent le serveur toutes les 1,5 secondes via un setInterval() et mettent à jour leur interface en fonction du statut reçu.

4. Fonctionnalités Développées

4.1 Système de score (mode Classique)

Situation	Points attribués
Bonne réponse très rapide (< 1 seconde)	**1000 points**
Bonne réponse lente (proche du timer)	**500 points minimum**

Série de 3 bonnes réponses consécutives ou plus 🔥	**+200 points bonus (streak)**
Dernière question de la partie	**Score de la question × 2**
Mauvaise réponse ou non réponse	**0 point - streak remis à zéro**

4.2 Système d'avatars multicouches

Chaque avatar "Bernard" est composé de 14 couches d'images superposées par z-index. Les couches sont chargées dynamiquement selon la configuration choisie par le joueur dans le lobby.

Z-index	Couche	Détail
z = 0	**Aura**	Image animée - z-index variable selon le type (devant ou derrière)
z = 5	**Cheveux arrière**	5 styles × jusqu'à 21 coupes × 4 couleurs
z = 10	**Peau**	15 couleurs disponibles (8 pour les anonymes)
z = 20	**T-shirt**	20 modèles × 4 couleurs (10 pour les anonymes)
z = 30	**Veste classique**	19 modèles × 4 couleurs
z = 31	**Costume spécial**	8 thèmes : neutre, métier, médiéval, pirate, halloween, noël...
z = 40	**Barbe**	11 modèles × 4 couleurs (5 pour les anonymes)
z = 41	**Moustache**	11 modèles × 4 couleurs
z = 50	**Bouche**	Fixe (1 modèle)
z = 51	**Yeux**	Fixe (1 modèle)
z = 52	**Sourcils**	Fixe (1 modèle)
z = 53	**Nez**	Fixe (1 modèle)
z = 54	**Cheveux avant**	Synchronisé avec la couche z=5
z = 70	**Effet visuel**	Arc-en-ciel (CSS), lévitation (CSS), effets image pour les types ≥3

4.3 Système de rôles

Quatre niveaux hiérarchiques gérés en base de données, vérifiés à chaque appel sensible via la fonction hasRole() définie dans db.php.

Rôle	Niveau	Accès
utilisateur	1	Jouer · Profil complet · Toutes les options d'avatar · Joker 50/50 · Statistiques
createur	2	Tout l'utilisateur + Créer, modifier, supprimer ses propres quiz
admin	3	Tout le créateur + Gérer les quiz de tous + Panel admin (gestion des rôles)
fondateur	4	Accès total + Seul à pouvoir nommer des admins + Inaltérable

4.4 Progression et statistiques

- 5 rangs basés sur les bonnes réponses cumulées : Merguez de Bronze (0) → Apprenti Bernard (50) → Expert des Quiz (150) → Maître des Questions (350) → Légende de Bernard (700)
- 2 quêtes de déblocage d'effets : Arc-en-Ciel (10 parties jouées), Lévitation (3 médailles d'or)
- Statistiques persistantes : parties jouées, bonnes/mauvaises réponses, précision en %, podiums 1re/2e/3e place
- Profil public consultable par les autres membres
- Bernard Favori sauvegardé en base et proposé automatiquement à chaque lobby

4.5 Autres fonctionnalités notables

- Tchat en temps réel avec raccourcis emoji (👏 😱 🤡) déclenchant des effets sonores sur l'écran hôte
- Bulles de message flottantes animées sur l'écran principal
- 3 awards calculés côté serveur en fin de partie : L'Éclair (meilleur temps moyen), La Tortue, Le Miraculé
- Podium final animé : apparition séquencée 3e → 2e → 1er, révélation du vainqueur, confettis
- Joker 50/50 (membres uniquement) : supprime 2 mauvaises réponses, 1 utilisation par partie
- Gestion de la bibliothèque de quiz : création, modification, duplication, visibilité public/privé

5. Sécurité Applicative

5.1 Mesures implémentées

Mots de passe chiffrés (bcrypt)

Tous les mots de passe sont hachés avec password_hash() en bcrypt avant insertion. Jamais stockés en clair, jamais affichés côté admin. La vérification se fait via password_verify() - irréversible.

Injection SQL - Requêtes préparées PDO

L'ensemble des requêtes SQL utilisent des paramètres liés (prepare + execute). Aucune concaténation directe de données utilisateur dans une requête SQL. La couche PDO assure l'échappement automatique.

XSS (Cross-Site Scripting)

Toutes les données utilisateur affichées dans le HTML sont systématiquement échappées via htmlspecialchars(). Les messages du tchat sont également assainis côté serveur avec htmlspecialchars() avant stockage.

Vérification serveur des réponses

La correction des réponses est recalculée côté serveur dans api_live.php en comparant answer_index avec correct_answer stocké dans le JSON de session. Le champ is_correct envoyé par le client est complètement ignoré - impossible de tricher via la console du navigateur.

Rate limiting sur le join

Un délai minimum de 3 secondes est imposé entre deux actions join depuis la même session PHP. Cela empêche un acteur malveillant de spammer des faux joueurs pour remplir une partie.

Credentials en variables d'environnement

Les accès base de données (hôte, nom, utilisateur, mot de passe) sont stockés dans un fichier .env exclu du dépôt Git via .gitignore. Le fichier .htaccess bloque également tout accès HTTP direct à .env et db.php.

Contrôle d'accès par rôle

Chaque page sensible vérifie la session et le rôle dès le premier appel PHP avec hasRole(). Un utilisateur non autorisé (ex: utilisateur simple sur admin_users.php) est immédiatement redirigé vers le dashboard sans aucune donnée exposée.

5.2 Tableau récapitulatif des risques

Menace	Contre-mesure	Priorité	Statut
Injection SQL	PDO + requêtes préparées sur toutes les requêtes	Critique	✓ Résolu
XSS	htmlspecialchars() sur toutes les sorties HTML	Critique	✓ Résolu
Exposition credentials	.env exclu du Git, accès HTTP bloqué par .htaccess	Critique	✓ Résolu
Triche sur les réponses	Recalcul côté serveur - champ client ignoré	Haute	✓ Résolu
Spam de faux joueurs	Rate limiting session (3s entre deux join)	Haute	✓ Résolu
Accès non autorisé	hasRole() en début de chaque page sensible	Haute	✓ Résolu
Mots de passe en clair	bcrypt irréversible via password_hash()	Critique	✓ Résolu
Sessions de jeu en production	Dossier sessions/ exclu du Git via .gitignore	Moyenne	✓ Résolu

6. Déploiement en Production

6.1 Environnement de production

Hébergeur	Alwaysdata (offre mutualisée)
Serveur web	Apache 2.4 avec mod_rewrite
Version PHP	8.4.19

Version MariaDB	11.4.9
URL de production	bernardquizz.alwaysdata.net
HTTPS	Forcé via redirection .htaccess (X-Forwarded-Proto)
Configuration DB	Fichier .env - jamais versionné

6.2 Configuration .htaccess

Le fichier .htaccess assure quatre rôles critiques :

- Blocage de l'indexation des répertoires (Options -Indexes)
- Refus d'accès HTTP direct aux fichiers sensibles (db.php, .env)
- Redirection forcée vers HTTPS via X-Forwarded-Proto
- Réécriture des URLs sans extension .php pour des URLs propres (ex: /dashboard au lieu de /dashboard.php)

6.3 Procédure de déploiement

- Développement en local sur environnement WAMP/MAMP
- Versionning Git avec .gitignore configuré (.env, sessions/*.json, desktop.ini)
- Upload des fichiers PHP, CSS/JS et assets via FTP/SFTP sur alwaysdata
- Configuration du fichier .env sur le serveur avec les credentials de production
- Tests fonctionnels en conditions réelles (parties complètes multi-joueurs)

7. Pistes d'Amélioration Futures

7.1 Améliorations techniques prioritaires

Amélioration	Bénéfice attendu	Complexité	Priorité
WebSockets (Ratchet/Node.js)	Temps réel vrai - latence nulle, charge serveur réduite de ~90% (suppression du polling)	**Élevée**	**Haute**
Sessions en base de données	Élimine les race conditions liées à l'écriture simultanée dans les fichiers JSON	**Moyenne**	**Moyenne**
Timer synchronisé côté serveur	Recalcul du temps restant à la reconnexion - évite le décalage si	**Moyenne**	**Moyenne**

	rechargement de page		
Tokens CSRF sur les formulaires	Protection contre les attaques Cross-Site Request Forgery sur les actions sensibles	**Faible**	**Faible**
Vérification email à l'inscription	Confirmer l'authenticité des comptes et permettre la récupération de mot de passe	**Faible**	**Faible**
Nettoyage automatique des sessions	Suppression des fichiers JSON de sessions expirées via un cron job côté serveur	**Faible**	**Faible**

7.2 Nouvelles fonctionnalités envisagées

- Types de questions supplémentaires - Vrai/Faux, réponse libre (saisie texte), QCM avec image dans les réponses, questions à plusieurs bonnes réponses
- Historique des parties - enregistrement en DB des résultats avec date, quiz joué, joueurs et scores finals
- Mode équipes - joueurs regroupés en équipes avec score cumulé par groupe
- Progressive Web App (PWA) - installation sur l'écran d'accueil mobile, mode offline partiel pour les joueurs
- Panel admin étendu - statistiques globales de la plateforme, modération du tchat, journal de connexion
- API RESTful structurée - refonte d'api_live.php avec des endpoints dédiés, codes HTTP appropriés, validation stricte des entrées
- Système de quiz publics notés - les joueurs peuvent noter les quiz auxquels ils ont participé

- Fin du dossier technique -